

Agent Runtime 技术全景

Agent Runtime – 状态与记忆管理(Memory)

关键问题

这个子领域要解决的核心矛盾是: **多轮对话/跨 Session 的上下文持久化**, 「**存储结构**」 vs 「**检索精度**」的取舍。

具体展开为三个子矛盾:

1. **存储策略的保真度取舍**: 原文存储(不丢失细节) vs LLM 摘要抽取(压缩率高但信息畸变) vs 结构化定型分类(可检索但丢失原始语境)
 2. **检索范式的维度取舍**: 纯向量语义搜索(语义模糊匹配) vs 混合检索(向量+全文+图,精度高但系统复杂) vs 分层按需加载(上下文经济但可能漏召回)
 3. **架构模式的耦合度取舍**: 独立记忆层 API(解耦但多一跳延迟) vs 有状态 Agent 进程(低延迟但紧耦合) vs 嵌入式后端(轻量但功能受限)
-

技术方向分类

方向1: Agent 记忆层框架 (Memory Layer as a Service)

独立于 Agent 框架的记忆中间层,提供 `add()` / `search()` / `update()` 等标准 API,Agent 通过 API 调用而非内置记忆。

核心思路: 将记忆管理从 Agent 推理中解耦,记忆层独立演进,Agent 框架只需调 API。同时承担记忆提取(LLM 驱-将原始对话自动转化为结构化记忆条目)、存储(向量+图+关系型)、检索(语义+时间衰减+重要性加权)三项职责。多数支持多后端存储、LLM 无关配置、多租户隔离。

代表项目:

- **mem0** (mem0ai, ~20k stars): 领域标杆。三层记忆分类(User/Session/Agent) + 双存储引擎(向量+Neo4j图) + 自动记忆提取/冲突更新/衰减。API-First 设计,Python/TS SDK。核心差异是「记忆关系图」-不仅存”用户喜欢 Python”,还存”用户喜欢 Python -[原因]-> 数据分析工作”,图遍历可在复杂推理场景中追溯关联记忆。开放问题:LLM 驱动的记忆提取在高频对话下的 API 成本控制。
- **memU** (NevaMind-AI, ~200 stars): 最结构化的记忆方案。三层编译架构 Resource(原始素材,不可变) -> Item(六类定型:profile/event/knowledge/behavior/skill/tool) -> Category(自动聚类文件夹+摘要+embedding)。独创 Tool Memory(跟踪工具调用成功率/耗时/成本,自动生成 when_to_use 建议)和 Salience 强化记忆(访问越多权重越高)。双检索策略可切换:RAG 模式(快速向量召回) vs LLM 模式(深度语义排序)。开放问题:三层编译在长会话下的 LLM 调用开销。

- **ReMe** (agentscope-ai, ~200 stars): AgentScope 生态的记忆工具包。多后端 (Chroma/FAISS + Neo4j + SQLite/PostgreSQL)、分层记忆(工作/情节/语义)、自动总结压缩、生命周期管理。核心竞争力是 AgentScope 生态内的无缝集成,但对跨框架支持有限。开放问题:跨框架集成的成熟度、大规模记忆下的检索性能。
- **TencentDB-Agent-Memory** (TencentCloud, ~300 stars): 腾讯云的差异化方案。核心卖点是全本地部署-记忆存储和检索完全在用户侧,无需云端服务,满足金融/医疗/政府等强合规场景。支持向量化检索、多模态记忆(代码+文档+对话)、记忆过期淘汰(LRU+时间衰减)。开放问题:多设备/多终端记忆同步机制缺失。

适用场景: 需为现有 Agent/Agent 框架快速接入记忆能力的场景;需要记忆与 Agent 框架解耦独立演进的团队;多租户企业级 Agent 平台。

局限: 独立记忆层的额外网络延迟(API 调用);记忆提取依赖 LLM,成本随规模线性增长;记忆层和 Agent 推理分离可能丢失「Agent 主动选择记忆」的灵活性。

方向2: 有状态 Agent 与操作系统级记忆管理

将记忆管理嵌入 Agent 进程内部,Agent 拥有独立的持久化状态,自主管理记忆的「页面调度」-类似操作系统的虚拟内存管理。

核心思路: 不把记忆视为外部数据库查询,而是视为 Agent 内部状态的一部分。Agent 拥有「记忆页表」,自主决定哪些记忆调入当前上下文工作集,哪些交换到持久化存储。记忆的读写、更新、遗忘都是 Agent 的主动性操作,不是外部 API 的被动响应。

代表项目:

- **letta** (letta-ai, ~16k stars): 学术先驱(前身 MemGPT)。核心创新是「虚拟上下文管理」-LLM 上下文窗口是「物理内存」,完整记忆是「虚拟内存」,Agent 自主决策分页调入/调出。Memory Block 模型(Persona/Human/Conversation/Archival) + Self-Editing Memory(Agent 在对话中自我更新记忆) + ADEPT 部署框架(REST+WebSocket)。核心差异:记忆不是被检索的「数据库」,而是 Agent 可以读写修改的「内存空间」-Agent 拥有记忆的完全主权。开放问题:极端长对话(1000+轮)中 LLM 页面调度决策质量退化风险。
- **powermem** (oceanbase, ~300 stars): OceanBase 数据库团队出品。将传统数据库的存储引擎基因(LSM-Tree、持久化、事务)注入 Agent 记忆,强调「数据可靠性」。记忆分层(工作/短期/长期) + 混合检索(dense+sparse+rerank) + 插件化集成(LangChain/LlamaIndex)。核心差异:不是纯 AI 记忆系统,而是数据库工程师用数据库方法解决的记忆持久化问题-写入吞吐、查询性能、数据可靠性优先。开放问题:Rust 核心与 Python FFI 边界的稳定性。

适用场景: 需要 Agent 持久化状态且重启后可恢复的场景;Agent Server/Agent 即服务的部署模式;对记忆的主权性和主动性要求高的应用(如个人 AI 助手)。

局限: Agent 进程与记忆状态紧耦合,部署和扩展复杂度高;LLM 驱动的自主记忆管理引入了「Agent 是否会忘记重要记忆」的不确定性;与传统无状态微服务架构范式冲突。

方向3: 混合索引与统一搜索引擎

试图在一个引擎内融合向量语义搜索、全文关键词检索和图结构查询三种搜索范式,避免 Agent 记忆层依赖多个独立存储系统的复杂性。

核心思路: Agent 的记忆检索需求天然混合的-“用户上周提到过的那个 Python 项目叫什么?” 需要时间过滤(上周)+语义搜索(Python 项目)+精确匹配(项目名)。单一检索范式(纯向量/纯关键词)无法满足,必须混合。

代表项目:

- **beads** (gastownhall, ~200 stars): Go 原生实现的统一搜索引擎。核心是三合一混合索引-HNSW(向量) + 倒排索引(全文) + 邻接表(图)在单一引擎中维护,提供统一查询语言同时表达三种搜索范式。可作为 Go 库嵌入 Agent 进程(嵌入式模式)或独立部署(服务模式)。核心差异:三个引擎合一而非三个独立系统,简化运维但牺牲了单个引擎的极致优化。开放问题:向量+全文+图的同步更新一致性和写入性能瓶颈。
- **OpenViking** (volcengine, ~3k stars): 字节跳动/火山引擎的「上下文数据库」(Context DB)。不是通用搜索引擎,而是专为 Agent 场景设计。特点:混合存储(向量+结构化+图一体) + 上下文窗口管理(智能选择最相关记忆片段注入 LLM 提示) + 分层记忆(工作/情节/语义,借鉴认知科学) + 工具调用结果缓存。高并发低延迟(毫秒级检索)是其工程优势-火山引擎内部 Agent 平台已验证。核心差异:最「完整」的 Agent 记忆方案,从存储引擎到认知模型到工具缓存全覆盖。开放问题:分布式扩展能力和大规模集群下的延迟一致性。
- **seekdb** (oceanbase, ~400 stars): OceanBase 团队的激进探索-在分布式 SQL 数据库中内建向量和全文搜索。不是另起引擎,而是在 OceanBase 的分布式底座(Paxos 共识、自动分片、异地多活)上扩展 AI 搜索能力, `ORDER BY vec_distance` 作为标准 SQL 语法。核心差异:不需要运维额外的向量数据库,一个 OceanBase 集群解决所有存储+搜索需求。但内建向量搜索的 QPS 和专用向量数据库(Milvus/Qdrant)的差距是核心疑问。

适用场景: 复杂 Agent 记忆检索需要跨语义、关键词和关系联合查询的场景;运维能力有限,希望减少基础设施组件数量的团队。

局限: 统一引擎在单个维度(纯向量搜索、纯全文搜索)上可能不及专用系统极致;混合查询优化器本身是复杂工程问题;新技术成熟度低,生产验证少(除 OpenViking 有火山内部验证)。

方向4: 知识检索平台 (RAG as Memory Backend)

通过 RAG 全流程平台为 Agent 提供「企业知识库即记忆」的能力,Agent 通过 RAG API 查询结构化和非结构化知识。

代表项目:

- **ragflow** (infiniflow, ~40k stars): 中文 RAG 生态标杆。核心武器是 DeepDoc 深度文档解析(版面分析->表格识别->图表理解->语义分块) + RAPTOR 递归摘要(文档分层聚类+层级摘要) + 混合检索(关键词+向量+知识图谱) + 自研 Infinity 向量引擎(Rust实现)。提供可视化 UI 完整 RAG Pipeline,非技术人员也能操作。核心差异:不是 Agent 记忆系统,而是可被 Agent 调用的「企业知识库」-Agent 记忆的知识来源而非记忆本身。开放问题:DeepDoc 对复杂中文表格(特别是无边框表格)的解析精度。

适用场景: Agent 需要基于大量企业内部文档做问答和决策;非技术用户需要配置知识库;中英文混合文档场景。

局限: 侧重文档知识检索而非对话记忆和用户偏好记忆;RAG 检索和 Agent 推理之间存在上下文传递的信息损失;文档更新后索引重建的时间成本。

方向5: 事件溯源与不可变日志

以不可变事件流的方式记录 Agent 的所有操作和状态变更,通过事件重放实现 Agent 状态恢复和决策溯源。

代表项目:

- **hindsight** (vectorize-io, ~100 stars): 面向 Agent 的事件溯源(Event Sourcing)引擎。所有状态变更以事件序列(而非 CRUD)记录,事件写入后不可修改删除,按时序索引快速检索。核心场景:Agent 决策溯源(回溯为何做了特定决策)、审计合规(全量不可变操作日志)、状态恢复(事件重放重建 Agent 状态)。核心差异:与 mem0/letta 等「主动记忆管理」不同,Hindsight 是「被动记录」-不做提取/摘要/检索优化,专注「完整记录」。开放问题:事件日志存储膨胀和保留策略。

适用场景: 高合规要求行业(金融/医疗)的 Agent 操作审计;需要回溯 Agent 决策路径的调试场景。

局限: 被动记录,不提供检索或记忆提取能力;仅保证记录完整性,不保证检索效率;不适合实时记忆查询场景。

方向6: 原文优先存储与结构化检索 (原文路线)

拒绝 LLM 摘要/抽取范式,坚持将原始对话原文逐字存储,仅通过层次化结构和语义检索实现记忆召回。核心论点是「LLM 摘要引入信息畸变,丢失关键细节,原文+精确检索比摘要+模糊检索更好」。

代表项目:

- **mempalace** (MemPalace, ~100 stars): 本分类最独特的方案。核心哲学**不做摘要、不抽取、不改写**-所有对话原文逐字保存。独创「记忆宫殿」结构(借鉴古希腊 Method of Loci 空间记忆术):Wing(人/项目) -> Room(话题) -> Drawer(原文对话,永不摘要) -> Closet(AAOK 压缩索引,指向原文)。四层记忆加载栈:L0(身份,~50 tokens 始终加载) -> L1(关键事实 AAOK,~120 tokens 始终加载) -> L2(当前会话 Room 召回) -> L3(全宫殿深度搜索)。LongMemEval 基准:96.6% R@5(原文模式,无 LLM)和 99%+(加 LLM Rerank),均为领域最高。核心差异:不用 LLM 做记忆,用结构化层次+语义检索,避免了 LLM 摘要的信息畸变和成本。开放问题:超长对话(数万轮)下的检索精度和存储成本;AAOK 有损压缩在上下文窗口持续扩大的趋势下的价值重新评估。

适用场景: 需要精确记忆细节的场景(代码项目、研究项目、合规审计);不希望记忆受 LLM 摘要质量波动影响;隐私敏感(全本地运行,零遥测)。

局限: 原文存储的存储成本随对话量线性增长;层次结构需要用户/Agent 主动组织(非自动);社区规模小(非专业开发者主导的独立项目);缺乏企业级多租户支持。

方向7: 后端基础设施 (Agent Application Backend)

为 Agent 应用提供一站式的数据基础设施-数据库 + 向量搜索 + 实时订阅 + 身份认证的完整底座,一整套后端满足 Agent 所有数据需求。

代表项目:

- **supabase** (supabase, ~80k stars): 开源 Firebase 替代。核心价值是在 PostgreSQL 底座上通过扩展实现 Agent 所有后端需求:pgvector(语义搜索) + PostgREST(自动 REST API) + Row Level Security(数据隔离) + Realtime(WebSocket 实时推送) + GoTrue(认证)。核心差异:不提供记忆管理逻辑,提供「可以构建记忆系统的基础设施」。Agent 开发者可以在 PostgreSQL 中同时管理结构化数据(用户配置)、向量数据(记忆语义检索)和时序数据(对话历史),一个 SQL 查询完成跨维度关联。开放问题:pgvector 在 10M+ 向量规模下 vs 专用向量数据库(Qdrant/Milvus)的性能差距。
- **NoKV** (NoKV-Lab, ~100 stars): 一个理论探索项目-挑战「所有持久化都需要 Key-Value 抽象」的假设。使用对象/文档原生存储,无预定义 Schema,让 Agent 状态以自然形状持久化。Rust 实现,轻量嵌入。核心差异:最轻量的存储方案,用于 Agent 原型的快速迭代。开放问题:与成熟 KV 引擎(RocksDB)的性能差距。

适用场景: 自建 Agent 平台需要完整后端基础设施的团队;已有 PostgreSQL 运维能力的团队;需要 SQL + 向量混合查询的场景。

局限: 不提供任何记忆管理逻辑(不提取/不摘要/不分类),仅为存储能力;pgvector 在超大规模下的性能天花板;不是开箱即用的 Agent 记忆方案。

当前趋势与开放争议

趋势

1. **从「外挂記憶庫」到「Agentic Memory」:** 记忆不再是 Agent 查一下的数据库,而是 Agent 可以感知、修改、遗忘的「自主记忆」-Letta 的 Self-Editing Memory 和 mempalace 的 Memory Agent (MCP 35 个记忆工具)代表了这一方向。
2. **混合检索成为标配:** 纯向量语义搜索在精确匹配和复杂关系查询上暴露短板,向量+全文+图(+时间衰减+重要性加权)的混合检索成为共识,只是实现方式不同(统一引擎 vs 多引擎组合)。
3. **本地优先与隐私需求崛起:** TencentDB-Agent-Memory 全本地部署、mempalace 零遥测默认本地、powermem 强调本地可靠性-「数据不离开用户机器」正在从 nice-to-have 变为企业合规 must-have。
4. **记忆性能基准开始出现:** LongMemEval 和 LoCoMo 等基准的推出让「谁的记忆检索更好」有了量化比较工具,mempalace 96.6% R@5 设立了原文路线的高标准。

开放争议

1. **原文 vs 摘要的永恒辩论:** mempalace 证明原文+层次结构在 LongMemEval 上 96.6% R@5 > 几乎所有摘要方案。但这是 500-2000 轮规模的基准,数万轮对话下原文存储的检索精度和存储成本是否仍然碾压摘要?摘要方案的核心优势是存储压缩和语义泛化-你不需要精确记住用户 3 个月前说的每一个字,只需要知道「用户偏好 Python」,摘要方案可能更经济。
2. **LLM 驱动的记忆提取成本与效益:** mem0/memU/ReMe 都依赖 LLM 从原始对话中提取和更新记忆。每次对话都调用 LLM 做记忆提取,API 成本是否可接受?在上下文窗口快速扩大(128k->1M tokens)的趋势下,是否直接把完整历史塞进上下文更经济?
3. **记忆层应独立还是内嵌:** mem0 的 API-First(独立记忆层) vs Letta 的 Stateful Agent(内嵌记忆)代表了两种哲学。独立记忆层的优势是解耦和独立演进;内嵌记忆的优势是低延迟和 Agent 对记忆的完全主权。最终答案可能是「两层都保留」-热记忆在 Agent 进程中,冷记忆在独立记忆层。
4. **数据库基因 vs AI 基因:** powermem(数据库工程师写的)和 mem0(AI 工程师写的)对「什么是好记忆」的定义不同。数据库基因优先可靠性/持久化/事务/写入性能;AI 基因优先检索精度/语义理解/LLM 集成。两类方案在工程实践上的融合可能是方向。
5. **记忆系统的可观测性缺失:** 几乎没有一个项目对记忆系统的「记忆命中率」「错误记忆注入率」「记忆漂移度」提供标准化评估工具。Agent 越用越聪明还是越用越偏?目前无法量化回答。

项目全景表

项目	方向	存储方式	核心差异	记忆分类	部署模式	活跃度
mem0	记忆层框架	向量DB + 图DB(Neo4j)	记忆关系图 + 自动提取 + 冲突更新	User/Session/Agent 三层	API云 + 本地	高(领域标杆)
letta	有状态 Agent	Memory Block + PostgreSQL	OS虚拟内存模型 + Self-Editing Memory	Persona/Human/Conversation/Archival	自托管 Server	高(学术先驱)
ragflow	RAG 知识平台	Infinity自研向量DB + ES	DeepDoc深度文档解析 + 可视化 UI	文档知识库	自托管 Docker	高 (~40k stars)
supabase	后端基础设施	PostgreSQL + pgvector	SQL+向量+实时+认证一站式	无(提供存储能力)	自托管/云	高 (~80k stars)
OpenViking	混合搜索引擎	向量+结构化+图混合引擎	专为Agent设计的Context DB	工作/情节/语义三层	自托管 + 集群	中(火山内部验证)
beads	混合搜索引擎	向量(HNSW)+全文+图一体	Go嵌入式三合一引擎	无(提供检索能力)	嵌入/独立服务	低
memU	记忆层框架	pgvector/SQLite	六类定型抽取 + Tool Memory + 自组织文件夹	Resource/Item/Category 三层	本地/服务端	低
ReMe	记忆层框架	Chroma/FAISS + Neo4j + PostgreSQL	AgentScope生态记忆工具包	工作/情节/语义三层	嵌入 Agent 框架	低
TencentDB-Agent-Memory	记忆层框架	ChromaDB/Milvus/FAISS	全本地部署 + 隐私优先	用户/会话/知识	全本地	低(早期)
powermem	有状态 Agent	LSM-Tree + 向量DB	数据库基因 + 事务持久化	工作/短期/长期三层	插件嵌入	低(早期)
mempalace	原文存储	ChromaDB/Qdrant/pgvector	纯原文+记忆宫殿 + LongMemEval 第一	Wing/Room/Drawer/Closet 四层	本地优先	低(独立项目)
hindsight	事件溯源	不可变事件日志	Event Sourcing + Agent操作审计	无(完整记录)	嵌入 Agent 进程	低
seekdb	混合搜索引擎	SQL+向量+全文一体分布式	分布式数据库内建AI搜索	无(提供搜索能力)	分布式集群	低(早期)
NoKV	后端基础	文档/对象原生存储	去KV化 + 轻量嵌入	无(提供存储能力)	嵌入 Agent	低

项目	方向	存储方式	核心差异	记忆分类	部署模式	活跃度
	设施				进程	

为团队选型的决策框架

根据团队在 Agent Memory 方向的实际需求,建议按以下维度评估:

如果要选一个记忆层直接集成到现有 Agent 系统

条件	推荐
需要快速接入、有云服务支持	mem0 (最成熟,API最简洁,社区最大)
需要全本地部署、数据不出企业	TencentDB-Agent-Memory 或 mempalace
需要最结构化的记忆分类和工具记忆	memU (六类定型 + Tool Memory)
已有 AgentScope 生态	ReMe (生态内无缝集成)

如果要自建 Agent 的有状态记忆架构

条件	推荐
想要 OS 级记忆管理、Agent 主权记忆	letta 架构思路(不一定用 Letta 本身)
需要数据库级可靠性、事务性	powermem 思路(LSM-Tree 持久化)
不需要 LLM 做记忆提取、想要精确原文检索	mempalace 的层次结构+原文路线

如果要搭建 Agent 应用的完整后端基础设施

条件	推荐
需要 SQL+向量+实时+认证全栈	supabase + pgvector (最成熟)
需要分布式+向量搜索一体化	seekdb (OceanBase 底座,但早期)
需要专为 Agent 设计的混合搜索引擎	OpenViking (火山内部已验证)

如果要把企业文档知识库接入 Agent

条件	推荐
中文文档为主、需要可视化	ragflow
需要 Agent 操作审计和决策溯源	hindsight (补充,非主记忆)